Chương 6 .

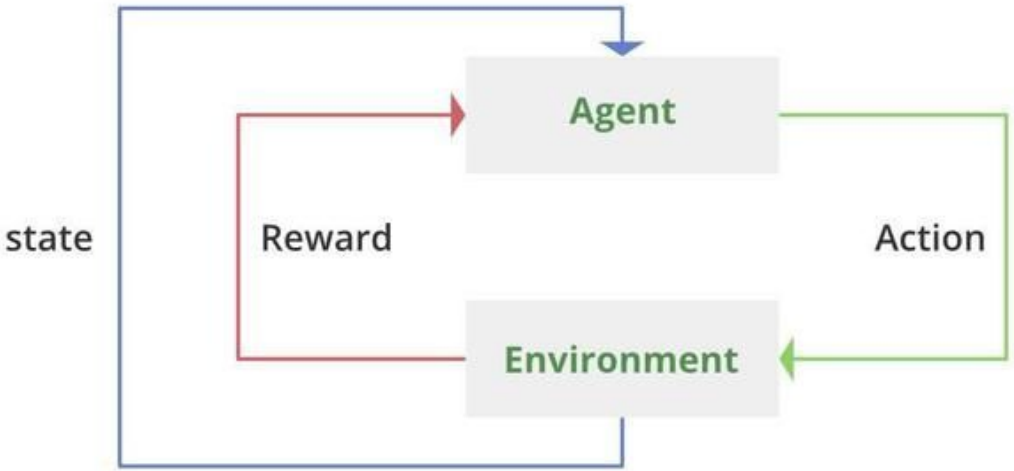
Học tăng cường

Ph.D. Nguyen Thi Khanh Tien

Học tăng cường (RL) là một kỹ thuật học máy trong đó tác nhân học cách đưa ra quyết định bằng cách tương tác với môi trường. Không giống như học có giám sát, cung cấp dữ liệu được dán nhãn, các tác nhân RL học từ hậu quả của hành động của họ.

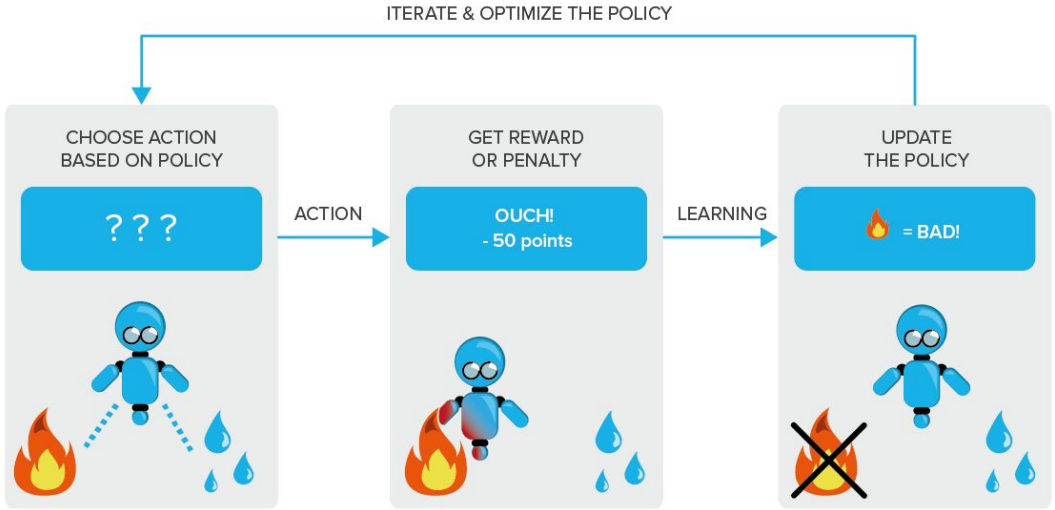
Thành phần cốt lõi của Học tăng cường

- **Tác nhân:** Người ra quyết định, người học cách tương tác với môi trường.
- **Môi trường:** Thế giới mà tác nhân hoạt động.
- **Trạng thái:** Hiện trạng hoặc điều kiện của môi trường.
- **Hành động:** Những lựa chọn mà tác nhân có thể thực hiện.
- **Phần thưởng:** Tín hiệu phản hồi cho biết sự thành công hay thất bại của một hành động.
- **Chính sách:** Chiến lược của tác nhân để lựa chọn hành động trong một trạng thái.



Học tăng cường hoạt động như thế nào

1. **Khởi tạo:** Tác nhân bắt đầu ở trạng thái ban đầu của môi trường.
2. **Lựa chọn hành động:** Tác nhân chọn một hành động dựa trên chính sách hiện tại của nó.
3. **Tương tác môi trường:** Tác nhân thực hiện hành động và môi trường chuyển sang trạng thái mới. 4. **Phần thưởng:** Đại lý nhận được phần thưởng dựa trên việc chuyển đổi.
5. **Đang học:** Đại lý cập nhật chính sách của mình dựa trên phần thưởng nhận được.
6. **Lặp lại:** Quá trình này được lặp đi lặp lại nhiều lần, với việc tác nhân dần dần cải thiện khả năng ra quyết định của mình.



Sự khác biệt giữa Học tăng cường và Học có giám sát

Reinforcement learning	Supervised learning
Reinforcement learning is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input	In Supervised learning, the decision is made on the initial input or the input given at the start
In Reinforcement learning decision is dependent, So we give labels to sequences of dependent decisions	In supervised learning the decisions are independent of each other so labels are given to each decision.
Example: Chess game,text summarization	Example: Object recognition,spam detetction

Các loại hình học tăng cường

Có hai loại RL

1. **Tích cực:** Củng cố tích cực được định nghĩa là khi một sự kiện xảy ra do một hành vi cụ thể, làm tăng cường độ và tần suất của hành vi đó. Nói cách khác, nó có tác động tích cực đến hành vi.

    Ưu điểm của học tăng cường là:

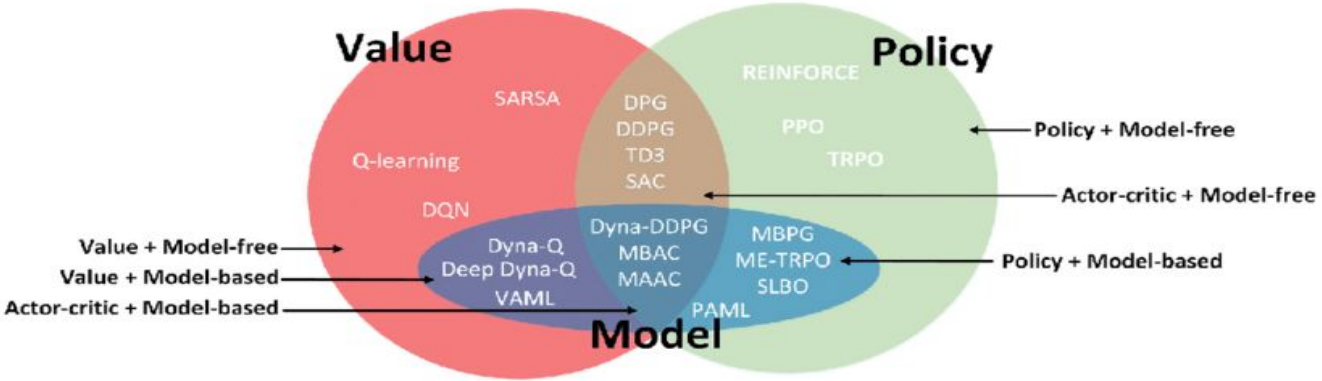
    - Tối đa hóa hiệu suất
    - Duy trì sự thay đổi trong thời gian dài
    - Quá nhiều Tăng cường có thể dẫn đến tình trạng quá tải và có thể làm giảm kết quả

2. **Tiêu cực:** Củng cố tiêu cực được định nghĩa là củng cố hành vi vì một điều kiện tiêu cực bị dừng hoặc tránh được.

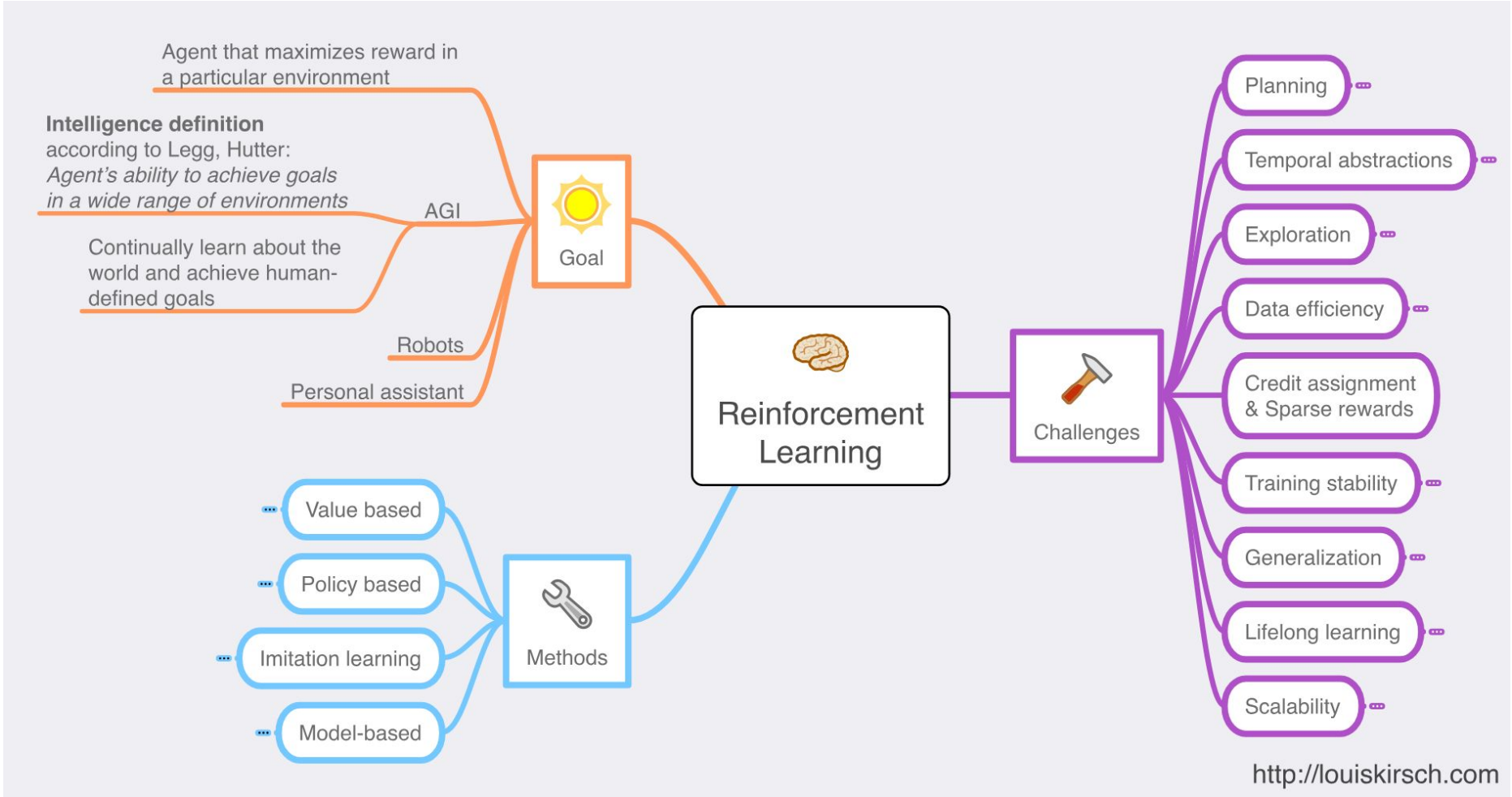
    Ưu điểm của học tăng cường:

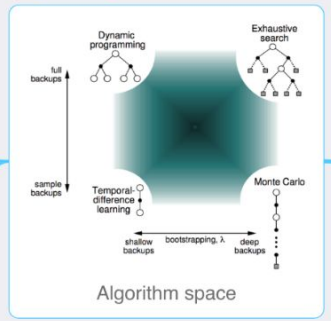
    - Tăng hành vi
    - Cung cấp sự thách thức đối với tiêu chuẩn hiệu suất tối thiểu
    - Nó chỉ cung cấp đủ để đáp ứng hành vi tối thiểu

Loại hình học tăng cường



- ***Phương pháp dựa trên giá trị***: Các phương pháp này ước tính giá trị của từng trạng thái hoặc cặp hành động trạng thái trong môi trường. Tác nhân tìm hiểu hành động nào tốt hơn bằng cách so sánh các giá trị. Ví dụ: Q-learning và Deep Q-Networks (DQN)
- ***Phương pháp dựa trên chính sách:*** Trong các phương pháp này, tác nhân trực tiếp tìm hiểu chính sách ánh xạ các trạng thái tới xác suất thực hiện từng hành động. Cách tiếp cận này hữu ích trong không gian hành động nhiều chiều hoặc liên tục. Các ví dụ bao gồm REINFORCE và các phương pháp gradient chính sách.
- ***Phương pháp dựa trên mô hình:*** Những phương pháp này liên quan đến việc xây dựng mô hình môi trường. Tác nhân tìm hiểu cách hoạt động của môi trường và đưa ra quyết định bằng cách sử dụng mô hình này. Cách tiếp cận này có thể hiệu quả hơn về mẫu nhưng đòi hỏi một mô hình môi trường tốt.


Agent that maximizes reward in a particular environment
Intelligence definition according to Legg, Hutter: Agent's ability to achieve goals in a wide range of environments
AGI
Continually learn about the world and achieve human-defined goals
Robots
Personal assistant
Goal
Reinforcement Learning
Challenges
Planning
Temporal abstractions
Exploration
Data efficiency
Credit assignment & Sparse rewards
Training stability
Generalization
Lifelong learning
Scalability
Value based
Policy based
Imitation learning
Model-based
Methods
http://louiskirsch.com

- Methods
  - Value based
    - Implementations
      - Q-Learning
      - Sarsa
      - Dyna
      - DDPG
    - 

      Algorithm space
      - Full backups vs sample backups (tree width)
      - Shallow backups vs deep backups (tree depth)
    - Advantages
      - More data efficient than policy based RL
    - Disadvantages
      - High dimensional / continuous action spaces
      - Stochastic policies
  - Policy based
    - Policy gradient
      - REINFORCE (Vanilla)
      - TRPO
      - PPO
      - A2C
      - A3C
      - Generalized Advantage Estimation (GAE)
      - TRPO Kronecker-factored (ACKTR)
      - ACER
      - SAC
    - Deterministic Policy Gradients
      - DPG
      - DDPG
      - TD3
    - Evolutionary
      - CEM
      - CMA-ES
      - PEPG
      - NES
      - ES
    - Policy search
      - PILCO
      - REPS
  - Imitation learning
    - DAGGER
    - GAIL
    - DeepMimic
    - VAIL
    - MetaMimic
  - Model-based
    - Pure planning
    - Expert Iteration
    - Data Augmentation
    - Embedding planning loops into policies
    - AIXI

http://louiskirsch.com

Ứng dụng của Học tăng cường

- **Robot:** Điều khiển robot để thực hiện các nhiệm vụ như nắm bắt đồ vật hoặc điều hướng môi trường.
- **Chơi trò chơi:** Phát triển các đại lý chơi các trò chơi như cờ vua, cờ vây và trò chơi điện tử.
- **Tài chính:** Giao dịch thuật toán, quản lý danh mục đầu tư.
- **Chăm sóc sức khỏe:** Kế hoạch điều trị được cá nhân hóa, khám phá thuốc.
- **Hệ thống đề xuất:** Đề xuất sản phẩm hoặc nội dung cho người dùng.

Các khái niệm chính

- **Thăm dò và Khai thác:** Tác nhân phải cân bằng việc khám phá các hành động mới để khám phá những phần thưởng tốt hơn với việc khai thác các hành động tốt đã biết để tối đa hóa phần thưởng ngay lập tức.
- **Quy trình ra quyết định Markov (MDP):** Một khung toán học để mô hình hóa các vấn đề ra quyết định tuần tự.
- **Hàm giá trị:** Ước tính phần thưởng dự kiến trong tương lai từ một trạng thái nhất định.
- **Q-Learning:** Một thuật toán RL phổ biến tìm hiểu phần thưởng dự kiến trong tương lai khi thực hiện một hành động ở một trạng thái nhất định.
- **Deep Q-Networks (DQN):** Kết hợp deep learning với Q-learning cho các vấn đề phức tạp.

Thách thức và định hướng tương lai

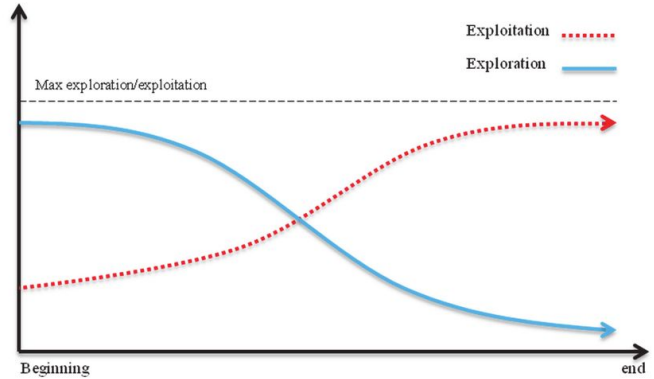
Mặc dù học tăng cường đã cho thấy những kết quả ấn tượng nhưng vẫn còn những thách thức cần vượt qua:

- **Hiệu quả mẫu:** Thuật toán RL thường yêu cầu một lượng lớn dữ liệu để học hiệu quả.
- **Tình thế tiến thoái lưỡng nan về thăm dò-khai thác:** Tìm kiếm sự cân bằng phù hợp giữa việc khám phá những hành động mới và khai thác những hành động tốt đã biết.
- **Chuyển nhượng tín dụng:** Việc xác định hành động nào đã đóng góp vào phần thưởng có thể khó khăn.

Nghiên cứu trong tương lai tập trung vào việc phát triển các thuật toán hiệu quả hơn, giải quyết hiệu quả của mẫu và áp dụng RL cho các vấn đề trong thế giới thực với độ phức tạp cao hơn.

Thăm dò và Khai thác: Đạo luật Cân bằng

Khám phá và khai thác là một thách thức cơ bản trong học tập tăng cường. Đó là việc đạt được sự cân bằng hợp lý giữa việc thử những điều mới (khám phá) và gắn bó với những gì hiệu quả (khai thác).

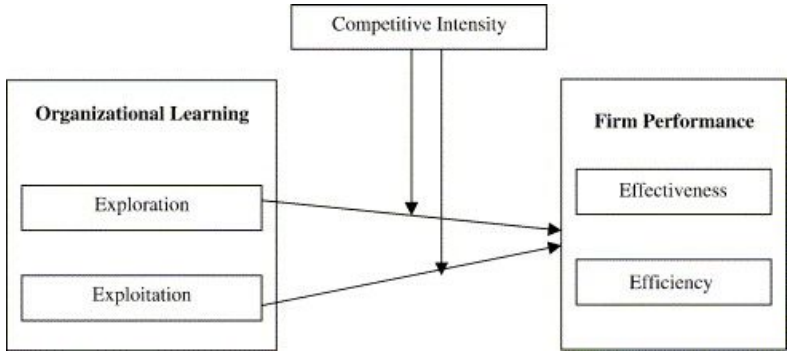


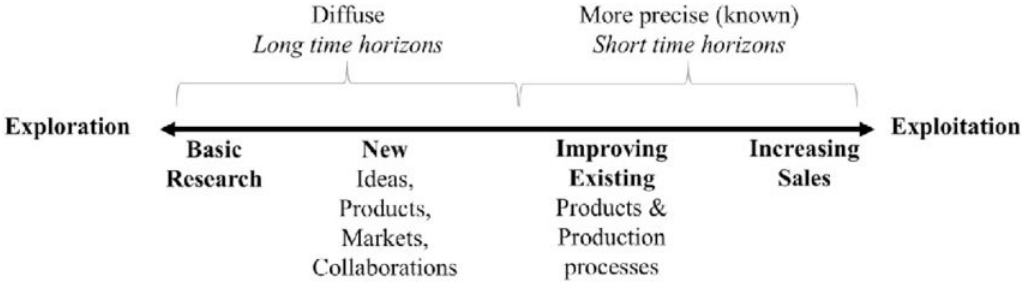
Thăm dò

- **Định nghĩa:** Thử các hành động hoặc chiến lược mới để khám phá các lựa chọn có khả năng tốt hơn.
- **Mục tiêu:** Tìm giải pháp tối ưu hoặc gần tối ưu bằng cách khám phá môi trường.
- **Rủi ro:** Có khả năng giảm phần thưởng ngắn hạn do không chắc chắn.

Khai thác

- **Định nghĩa:** Sử dụng chiến lược nổi tiếng nhất hiện nay để tối đa hóa phần thưởng ngay lập tức.
- **Mục tiêu:** Tối đa hóa hiệu suất dựa trên kiến thức hiện tại.
- **Rủi ro:** Bị mắc kẹt trong điểm tối ưu cục bộ và bỏ lỡ những khả năng tốt hơn.





Sự tiến thoái lưỡng nan

Một tác nhân chỉ khai thác kiến thức hiện tại của mình sẽ không bao giờ khám phá được các lựa chọn tốt hơn. Ngược lại, một tác nhân chỉ khám phá có thể không bao giờ đạt được phần thưởng đáng kể. Điều quan trọng nằm ở việc tìm ra sự cân bằng phù hợp giữa cả hai.

Chiến lược cân bằng thăm dò và khai thác

Một số kỹ thuật có thể giúp nhân viên giải quyết tình huống khó xử này:

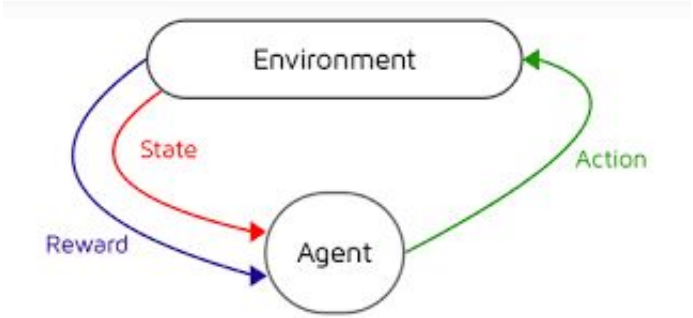
- **Tham lam Epsilon:** Một cách tiếp cận đơn giản trong đó tác nhân chọn một hành động ngẫu nhiên với xác suất epsilon và hành động nổi tiếng nhất với xác suất 1-epsilon.
- **Giới hạn tin cậy trên (UCB):** Phương pháp này cân bằng giữa việc thăm dò và khai thác bằng cách xem xét cả phần thưởng trung bình và số lần một hành động đã được thử.
- **Lấy mẫu Thompson:** Xử lý vấn đề như một vấn đề suy luận Bayes, gán xác suất cho các hành động khác nhau và chọn hành động dựa trên những xác suất này.
- **Thuật toán học tăng cường:** Nhiều thuật toán RL vốn đã cân bằng giữa việc thăm dò và khai thác, chẳng hạn như các phương pháp Deep Q-Networks (DQN) và Policy gradient.

Ví dụ: Hệ thống gợi ý nhà hàng

- **Khám phá:** Đề xuất một nhà hàng mới cho người dùng, ngay cả khi sở thích trước đây của người dùng gợi ý một loại hình ẩm thực khác.
- **Khai thác:** Đề xuất một nhà hàng tương tự như địa điểm yêu thích trước đây của người dùng.

Quy trình Quyết định Markov (MDP)

Quy trình ra quyết định Markov (MDP) là một khung toán học để lập mô hình việc ra quyết định trong các tình huống trong đó kết quả một phần ngẫu nhiên và một phần nằm dưới sự kiểm soát của người ra quyết định. Đó là nền tảng trong học tập tăng cường.



Các thành phần của MDP

- **Trạng thái (S):** Tất cả các tình huống hoặc điều kiện có thể xảy ra mà đại lý có thể gặp phải.
- **Hành động (A):** Tập hợp các hành động có thể có mà tác nhân có thể thực hiện trong một trạng thái nhất định.
- **Xác suất chuyển tiếp (P):** Xác suất chuyển từ trạng thái này sang trạng thái khác sau khi thực hiện một hành động cụ thể.
- **Phần thưởng (R):** Giá trị ngay lập tức mà tác nhân nhận được khi chuyển từ trạng thái này sang trạng thái khác và thực hiện một hành động cụ thể.
- **Chính sách (π):** Chính sách là một chiến lược chỉ rõ hành động cần thực hiện ở mỗi trạng thái. Một chính sách có thể mang tính quyết định, trong đó một hành động cụ thể được chọn cho từng trạng thái hoặc ngẫu nhiên, trong đó phân bổ xác suất cho các hành động được xác định cho từng trạng thái.
- **Hệ số chiết khấu (γ):** Giá trị từ 0 đến 1 xác định tầm quan trọng của phần thưởng trong tương lai.

States:	**S**
Model:	**T(S, a, S') ~ P(S' | S, a)**
Actions:	**A(S), A**
Reward:	**R(S), R(S, a), R(S, a, S')**
Policy:	**$\prod(S) \rightarrow a$** **$\prod^*$**

Markov Decision Process

Giải quyết quá trình ra quyết định Markov

Có một số phương pháp để giải quyết MDP, bao gồm:

- **Lặp giá trị:** Một thuật toán lặp cập nhật giá trị của từng trạng thái với giả định hoạt động tối ưu từ trạng thái đó trở đi. Thuật toán cập nhật lặp đi lặp lại giá trị của từng trạng thái cho đến khi các giá trị hội tụ về một tập hợp ổn định, đại diện cho hàm giá trị của chính sách tối ưu.
- **Lặp lại chính sách:** Một thuật toán lặp xen kẽ giữa đánh giá chính sách (tính toán giá trị của chính sách) và cải tiến chính sách (sử dụng hàm giá trị để tìm chính sách tốt hơn). Quá trình này được lặp lại cho đến khi chính sách hội tụ về chính sách tối ưu.

- **Q-Learning:** Thuật toán học tăng cường không có mô hình giúp tìm hiểu chất lượng của hành động (ký hiệu là giá trị Q) trực tiếp từ trải nghiệm mà không cần mô hình động lực học của môi trường. Q-learning có thể tìm ra chính sách tối ưu ngay cả khi chưa xác định được mô hình của môi trường.

Ứng dụng của Quy trình Quyết định Markov

MDP có nhiều ứng dụng, bao gồm:

- Robotics để lập kế hoạch đường đi và tránh chướng ngại vật.
- Quản lý tài nguyên trong hệ thống máy tính và mạng.
- Các mô hình tài chính để tối ưu hóa đầu tư và danh mục đầu tư.
- Lý thuyết trò chơi và ra quyết định trong môi trường cạnh tranh.
- Lập kế hoạch điều trị y tế và tối ưu hóa chăm sóc bệnh nhân.

Trạng thái là tập hợp các mã thông báo đại diện cho mọi trạng thái mà tác nhân có thể ở đó.

Một **Mô hình** (đôi khi được gọi là Mô hình chuyển tiếp) đưa ra hiệu ứng của một hành động trong một trạng thái. Cụ thể, T(S, a, S') xác định một chuyển tiếp T trong đó đang ở trạng thái S và thực hiện hành động 'a' sẽ đưa chúng ta đến trạng thái S' (S và S' có thể giống nhau). Đối với các hành động ngẫu nhiên (ồn ào, không xác định), chúng tôi cũng xác định xác suất P(S'|S,a) đại diện cho xác suất đạt đến trạng thái S' nếu hành động 'a' được thực hiện ở trạng thái S. Lưu ý Thuộc tính Markov cho biết rằng tác động của một hành động được thực hiện trong một trạng thái chỉ phụ thuộc vào trạng thái đó chứ không phụ thuộc vào lịch sử trước đó.

Hành động A là tập hợp tất cả các hành động có thể xảy ra. A(s) xác định tập hợp các hành động có thể được thực hiện ở trạng thái S.

Phần thưởng là chức năng phần thưởng có giá trị thực. R(s) biểu thị phần thưởng khi chỉ ở trạng thái S. R(S,a) biểu thị phần thưởng khi ở trạng thái S và thực hiện hành động 'a'. R(S,a,S') biểu thị phần thưởng khi ở trạng thái S, thực hiện hành động 'a' và kết thúc ở trạng thái S'.

Chính sách là một giải pháp cho Quy trình Quyết định Markov. Chính sách là ánh xạ từ S tới a. Nó chỉ ra hành động 'a' được thực hiện khi ở trạng thái S.

States:	**S**
Model:	**T(S, a, S') ~ P(S' | S, a)**
Actions:	**A(S), A**
Reward:	**R(S), R(S, a), R(S, a, S')**
Policy:	**$\Pi(S) \rightarrow a$** **Π^***

Markov Decision Process

Ví dụ về thế giới lưới

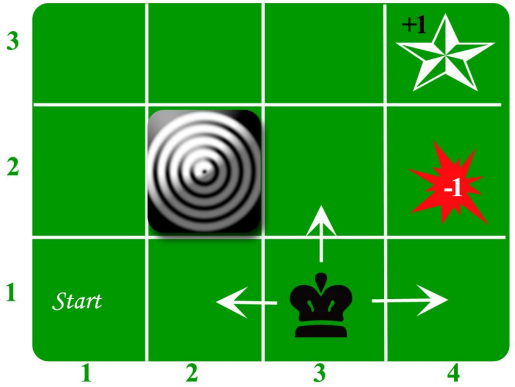
Một tác nhân sống trong lưới (ví dụ: 3*4).

Lưới có trạng thái BẮT ĐẦU (lưới (1,1)).

Mục đích của đặc vụ là đi lang thang quanh lưới để cuối cùng đến được Viên kim cương xanh (lưới (4,3)).

Trong mọi trường hợp, đặc vụ nên tránh lưới Lửa (màu cam, lưới (4,2)). Ngoài ra, lưới (2,2) là một lưới bị chặn, nó hoạt động như một bức tường nên tác nhân không thể vào được.

Tác nhân có thể thực hiện bất kỳ hành động nào sau đây: **LÊN, XUỐNG, TRÁI, PHẢI**



Các bức tường chặn đường đi của đặc vụ, tức là nếu có một bức tường theo hướng mà đặc vụ sẽ đi, thì đặc vụ vẫn ở nguyên vị trí. Vì vậy, ví dụ: nếu tác nhân nói TRÁI trong lưới BẮT ĐẦU, anh ta sẽ vẫn được đặt trong lưới BẮT ĐẦU.

Mục tiêu đầu tiên: Để tìm trình tự ngắn nhất đi từ BẮT ĐẦU đến Kim cương. Hai trình tự như vậy có thể được tìm thấy:

- **PHẢI- PHẢI- LÊN- LÊN-PHẢI**
- **LÊN- LÊN- PHẢI- PHẢI- PHẢI**

Chúng ta hãy lấy cái thứ hai (LÊN LÊN PHẢI PHẢI) cho cuộc thảo luận tiếp theo.
Việc di chuyển bây giờ ồn ào. 80% thời gian hành động dự định hoạt động chính xác. 20% thời gian mà tác nhân hành động thực hiện khiến nó chuyển động vuông góc.

Ví dụ: nếu tác nhân nói LÊN thì xác suất đi LÊN là 0,8 trong khi xác suất đi TRÁI là 0,1 và xác suất đi PHẢI là 0,1 (vì TRÁI và PHẢI là góc vuông với LÊN).

Đại lý nhận được phần thưởng cho mỗi bước thời gian: -

- Phần thưởng nhỏ cho mỗi bước (có thể âm khi cũng có thể coi là hình phạt, trong ví dụ trên vào Lửa có thể có phần thưởng là -1).

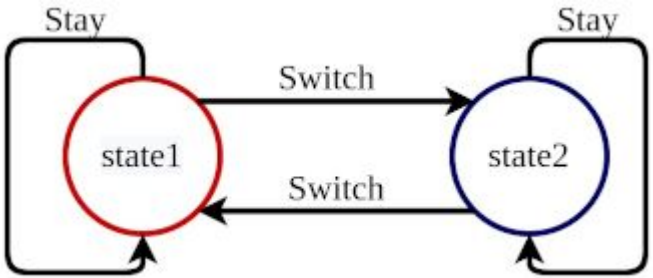
- Phần thưởng lớn sẽ đến vào lúc cuối (tốt hay xấu).
- Mục tiêu là Tối đa hóa tổng số phần thưởng.

Ví dụ mã hóa MDP đơn giản - MDP hai trạng thái

Đây là ví dụ về cách triển khai Quy trình Quyết định Markov (MDP) hai trạng thái. Nó chỉ có hai trạng thái: trạng thái 1 và trạng thái 2. Các ứng dụng MDP thường xuất hiện trong học máy và học tăng cường.

Ở bất kỳ giai đoạn nào, đại lý có thể thực hiện bất kỳ hành động nào trong hai hành động. Các hành động là ở lại hoặc chuyển đổi. Công tắc sẽ thay đổi trạng thái hiện tại sang trạng thái khác và sẽ không thực hiện bất kỳ thay đổi nào.

Tại bất kỳ thời điểm nào, chúng ta có thể in ra trạng thái hiện tại của hệ thống. Sơ đồ chuyển trạng thái có thể được hình dung như dưới đây.




```python
class TwoStateMDP(object):
    """docstring for TwoState"""
    def __init__(self):
        self.states = 2
        self.nactions = 2
        self.currentS = 1

    def stay(self):
        pass

    def toggle(self):
        if self.currentS ==1:
            self.currentS=2
        else:
            self.currentS=1

switch = TwoStateMDP()

print(switch.currentS)
switch.toggle()
print(switch.currentS)
switch.toggle()
print(switch.currentS)
```

Lệnh *'switch = TwoStateMDP()'* tạo ra một đối tượng thuộc loại MDP hai trạng thái. Theo mặc định, nó sẽ ở trạng thái 1. Khi chúng ta gọi hàm *'toggle()'*, nó sẽ được chuyển sang trạng thái khác.

Chức năng giá trị trong học tập tăng cường

Hàm giá trị trong học tăng cường là ước tính mức độ tốt của một tác nhân ở một trạng thái cụ thể hoặc thực hiện một hành động cụ thể trong một trạng thái nhất định. Nói một cách đơn giản hơn, đó là dự đoán về phần thưởng dự kiến trong tương lai mà một tác nhân có thể nhận được từ trạng thái hoặc hành động đó.

Có hai loại hàm giá trị chính: Hàm giá trị trạng thái (V), Hàm giá trị hành động (Q)

- The value function $V^{\pi}(s)$ assigns each state the expected reward

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{a_t, a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s \right]$$

- Usually not informative enough to make decisions.
- The Q-value $Q^{\pi}(s, a)$ is the expected reward of taking action a in state s and then continuing according to π.

$$Q^{\pi}(s, a) = \mathop{\mathbb{E}}_{a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s, a_t = a \right]$$

Vai trò của Chức năng Giá trị trong Học tập Tăng cường

- **Ra quyết định:** Tác nhân sử dụng các hàm giá trị để quyết định hành động nào sẽ thực hiện. Họ chọn những hành động dẫn đến trạng thái có lợi nhuận kỳ vọng cao hơn.
- **Học tập:** Các chức năng giá trị được học thông qua trải nghiệm. Các thuật toán học tăng cường cập nhật các ước tính hàm giá trị dựa trên sự tương tác của tác nhân với môi trường.
- **Cải thiện chính sách:** Bằng cách ước tính giá trị của các trạng thái và hành động khác nhau, đại lý có thể cải thiện chính sách của mình theo thời gian.

Các khái niệm chính liên quan đến hàm giá trị

- **Hệ số chiết khấu (γ):** Giá trị từ 0 đến 1 xác định tầm quan trọng của phần thưởng trong tương lai. Hệ số chiết khấu cao hơn sẽ mang lại nhiều trọng lượng hơn cho phần thưởng trong tương lai.
- **Phương trình Bellman:** Một phương trình cơ bản trong học tăng cường liên hệ giá trị của một trạng thái với giá trị mong đợi của các trạng thái kế tiếp nó.
- **Lập trình động:** Một kỹ thuật toán học được sử dụng để giải phương trình Bellman và tính toán các hàm giá trị tối ưu.

- **Học theo sự khác biệt theo thời gian:** Một phương pháp học tăng cường học các chức năng giá trị trực tiếp từ kinh nghiệm mà không cần dựa vào mô hình hoàn chỉnh của môi trường.

phương trình Bellman

- The foundation of many RL algorithms

$$
\begin{aligned}
V^{\pi}(s) &= \mathop{\mathbb{E}}_{a_t, a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s \right] \\
&= \mathop{\mathbb{E}}_{a_t} [r_t | s_t = s] + \gamma \mathop{\mathbb{E}}_{a_t, a_{t+i}, s_{t+i}} \left[\sum_{i=1}^{\infty} \gamma^i r_{t+i+1} | s_t = s \right] \\
&= \mathop{\mathbb{E}}_{a_t} [r_t | s_t = s] + \gamma \mathop{\mathbb{E}}_{s_{t+1}} [V^{\pi}(s_{t+1}) | s_t = s] \\
&= \sum_{a,r} P^{\pi}(a|s_t) p(r|a, s_t) \cdot r + \gamma \sum_{a,s'} P^{\pi}(a|s_t) p(s'|a, s_t) \cdot V^{\pi}(s')
\end{aligned}
$$

- Similar equation holds for Q

$$
\begin{aligned}
Q^{\pi}(s, a) &= \mathop{\mathbb{E}}_{a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s, a_t = a \right] \\
&= \sum_{r} p(r|a, s_t) \cdot r + \gamma \sum_{s'} p(s'|a, s_t) \cdot V^{\pi}(s') \\
&= \sum_{r} p(r|a, s_t) \cdot r + \gamma \sum_{a', s'} p(s'|a, s_t) p(a'|s') \cdot Q^{\pi}(s', a')
\end{aligned}
$$

Phương trình Bellman tối ưu

- First step is understand the Bellman equation for the optimal policy π^*
- Under this policy $V^*(s) = \max_a Q^*(s, a)$

$$
\begin{aligned}
V^*(s) &= \max_a \left[\mathbb{E}\left[r_{t+1} | s_t = s, a_t = a \right] + \gamma \mathop{\mathbb{E}}_{s_{t+1}} \left[V^*(s_{t+1}) | s_t = s, a_t = a \right] \right] \\
&= \max_a \left[\sum_r p(r|a, s_t) \cdot r + \gamma \sum_{s'} p(s'|a, s_t) \cdot V^*(s') \right] \\
Q^*(s, a) &= \mathbb{E}\left[r_{t+1} | s_t = s, a_t = a \right] + \gamma \mathop{\mathbb{E}}_{s_{t+1}} \left[\max_{a'} Q^*(s_{t+1}, a') | s_t = s, a_t = a \right] \\
&= \sum_r p(r|a, s_t) \cdot r + \gamma \sum_{s'} p(s'|a, s_t) \cdot \max_{a'} Q^*(s', a')
\end{aligned}
$$

- Set on nonlinear equations.
- Same issues as before.

Q-Learning: Học từ hành động

- Q-learning is a simple algorithm to find the optimal policy without knowing the model.
- Q^* is the unique solution to the optimal Bellman equation.

$$Q^*(s,a) = \mathbb{E}\left[r_{t+1}|s_t = s, a_t = a\right] + \gamma \mathop{\mathbb{E}}_{s_{t+1}}\left[\max_{a'} Q^*(s_{t+1}, a')|s_t = s, a_t = a\right]$$

- We don't know the model and don't want to update all states simultaneously.
- Solution - given sample $s_t, a_t, r_{t+1}, s_{t+1}$ from the environment update your Q-values so they are closer to satisfying the bellman equation.
  - off-policy method: Samples don't have to be from the optimal policy.
- Samples need to be diverse enough to see everything - exploration.

- Given Q-value the best thing we can do (given our limited knowledge) is to take $a = \arg\max_{a'} Q(s, a')$ - exploitation
- How do we balance exploration with exploitation?
- Simplest solution: ϵ-greedy.
  - With probability $1 - \epsilon$ pick $a = \arg\max_{a'} Q(s, a')$ (i.e. greedy)
  - With probability ϵ pick any other action uniformly.
- Another idea - softmax using Q values
  - With probability $1 - \epsilon$ pick $a = \arg\max_{a'} Q(s, a')$ (i.e. greedy)
  - With probability ϵ pick any other action with probability $\propto \exp(\beta Q(s, a))$.
- Other fancier solutions exist, many leading methods use simple ϵ-greedy sampling.

Thuật toán Q-learning

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
    $S \leftarrow S'$;
  until S is terminal

- Can prove convergence to the optimal Q^* under mild conditions.
- Update is equivalent to gradient descent on loss $||R + \gamma \max_a Q(S',a) - Q(s,a)||^2$.
- Why L_2 loss? Optimal solution is the mean which is what we are looking for!

Ví dụ: →
https://ww w.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial

Khởi động

- Another way to think about Q-learning.
- $Q(s, a)$ is the expected reward, can use Monte-Carlo estimation.
- Problem - you update only after the episode ends, can be very long (or infinite).
- Q-learning solution - take only 1 step forward and estimate the future using our Q value - bootstrapping.
  - "learn a guess from a guess"
- Q-learning is just one algorithm in a family of algorithms that use this idea.

Xấp xỉ hàm

- Q-learning still scales badly with large state spaces, how many states does a chess game have? Need to save the full table!
- Similar states, e.g. move all chess pieces two steps to the left, at treated as totally different.
- Solution: Instead of Q being a $S \times A$ table it is a parametrized function.
- Looking for function $\hat{Q}(s, a; \mathbf{w}) \approx Q^*(s, a)$
  - Linear functions $Q(s, a; \mathbf{w}) = \mathbf{w}^T \phi(s, a)$.
  - Neural network
- Hopefully can generalize to unseen states.
- Problem: Each change to parameters changes all states/actions - can lead to instability.
- For non-linear Q-learning can diverge.

Học Q sâu

- We have a function approximator $Q(s, a; \theta)$, standard is neural net but doesn't have to be.
- What is the objective that we are optimizing?
- We want to minimize $\mathbb{E}_\rho[||R + \gamma \max_{a'} Q(S', a') - Q(s, a)||^2]$
  - ρ is a distribution over states, depends on θ!
- Two terms depend on Q, don't want to take gradients w.r. to $\gamma \max_a Q(S', a)$
- We want to correct our previous estimation given the new information.

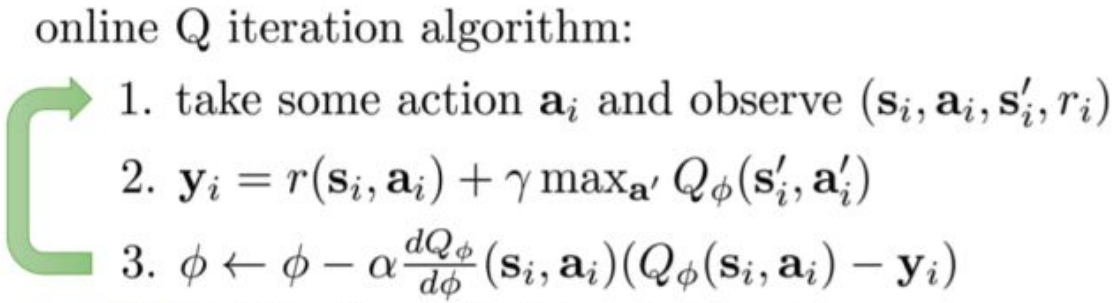


Figure: Take from:rll.berkeley.edu/deeprlcourse

- This simple approach doesn't work well as is.

Vấn đề và giải pháp

- Problem: data in the minibatch is highly correlated
  - Consecutive samples are from the same episode and probably similar states.
  - Solution: Replay memory.
  - You store a large memory buffer of previous (s, a, r, s') (notice this is all you need for Q-learning) and sample from it to get diverse minibatch.
- Problem: The data distribution keeps changing
  - Since we aren't optimizing y_i its like solving a different (but related) least squares each iteration.
  - We can stabilize by fixing a target network for a few iterations

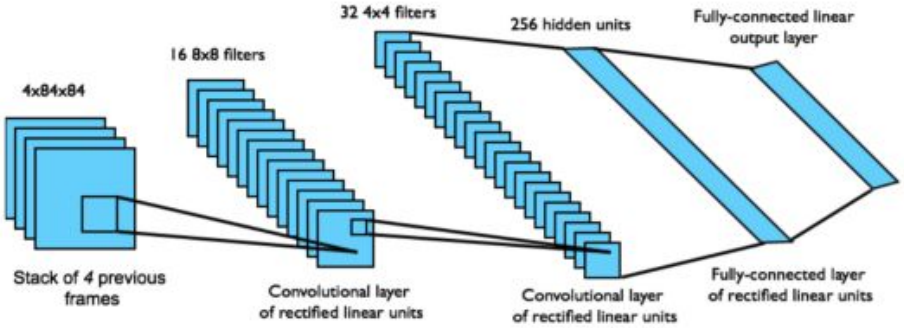


1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
3. compute $y_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using *target* network $Q_{\phi'}$
4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
5. update ϕ': copy ϕ every N steps

Figure: Take from:rll.berkeley.edu/deeprlcourse

Ví dụ. DQN trên trò chơi Atari

- Trained a NN from scratch on atari games



- Ablation study

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

tóm tắt RL

- Learning from experience not from labeled examples.
- Why is RL hard?
  - Limited feedback.
  - Delayed rewards.
  - Your model effect what you see.
  - Huge state space.
- Usually solved by learning the value function or optimizing the policy (not covered)
- Model based method but less successful at the moment.
- How do you define the rewards? Can be trick.
  - Bad rewards can lead to reward hacking

Tóm tắt Q-Learning

- Try to find Q that satisfies the optimal Bellman conditions
- Off-policy algorithm - Doesn't have to follow a greedy policy to evaluate it.
- Model free algorithm - Doesn't have any model for instantaneous reward or dynamics.
- Learns a separate value for each s, a pair - doesn't scale up to huge state spaces.
- Can scale using a function approximation
  - No more theoretical guarantees.
  - Can diverge.
  - Some simple tricks help a lot.

Học tăng cường với Q-learning và OpenAI Gym

Lắp đặt và thiết lập phòng tập thể dục

Tạo môi trường tùy chỉnh

Triển khai thuật toán học tăng cường

Hình dung quá trình đào tạo và kết quả

Tài liệu tham khảo

1. https://www.learndatasci.com/tutorials/reinforcement-q-learning-scratch-python-openai-gym/ 2. https://medium.com/swlh/introduction-to-q-learning-with-openai-gym-2d794da10f3d 3. https://github.com/srnand/Reinforcement-Learning-USE-OpenAI-Gym/tree/master/Mountain_Car 4. https://gymnasium.farama.org/